HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY
School of Information and Communication Technology

CTF CHALLENGE WRITE-UP

Web Exploitation

BKSEC: ezy_pzy

DucTapCodeDao 202416788

Mục lục

1 Phân tích Challenge **3**

2 Nhập payload và khảo sát Burp Suite **4**

3 Thay đổi JWT để lấy quyền admin **6**

4 Loại bỏ hướng OS Command Injection và tập trung vào SSRF **7**

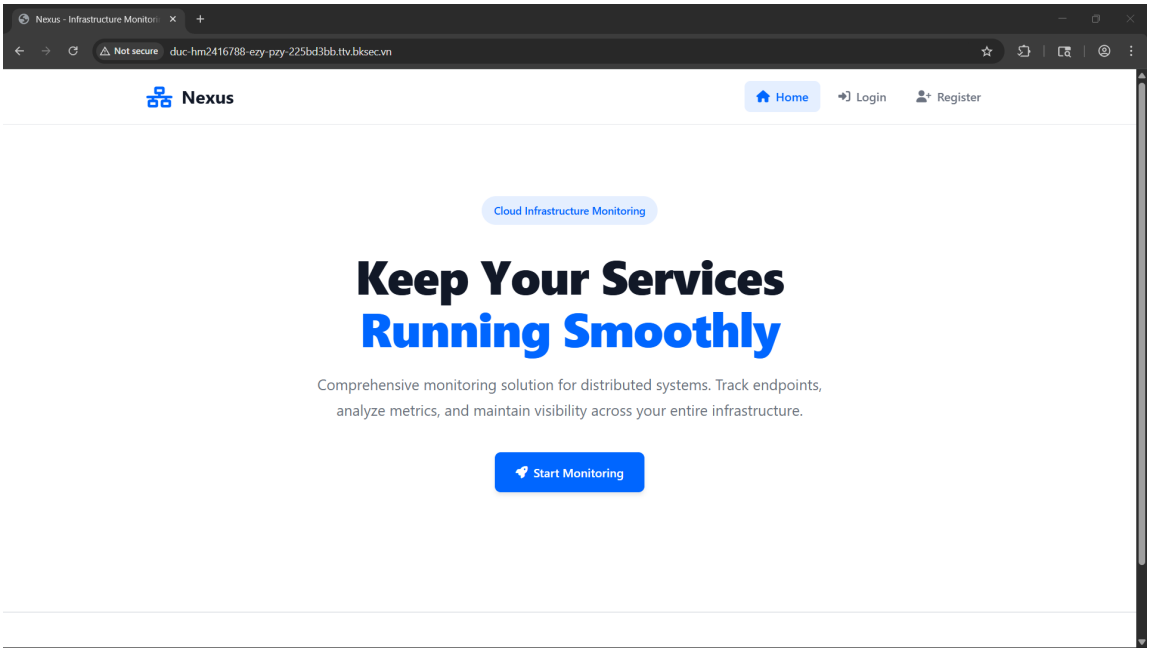
5 Khai thác SSRF để truy cập CouchDB và lấy flag **8**

1 Phân tích Challenge

Challenge cho chúng ta một đường link cùng với một đoạn gợi ý:

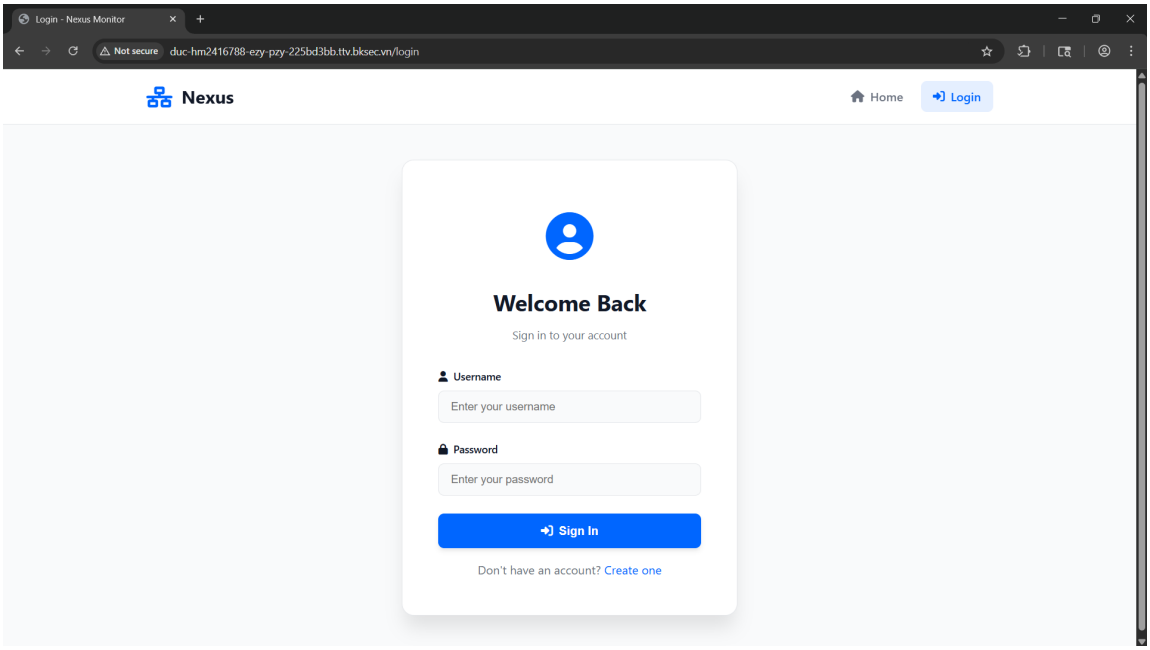
"This challenge is a piece of cake. Gudluk!"

Trước tiên, em mở trang web bằng trình duyệt *Google Chrome*, và đây là giao diện của trang web:



Hình 1: Giao diện khi mở đường link ở trình duyệt thông thường (Google Chrome).

Em bấm vào slide Login thì trang web chuyển sang giao diện này:



Hình 2: Giao diện khi chọn Login.

Nhìn vào giao diện, em nhận thấy đây là trang *Login* của hệ thống **Nexus** với hai trường nhập liệu là *Username* và *Password*, kèm theo nút *Sign In*.

Dự đoán ban đầu của em là challenge này có thể là 1 challenge về **SQL Injection**.

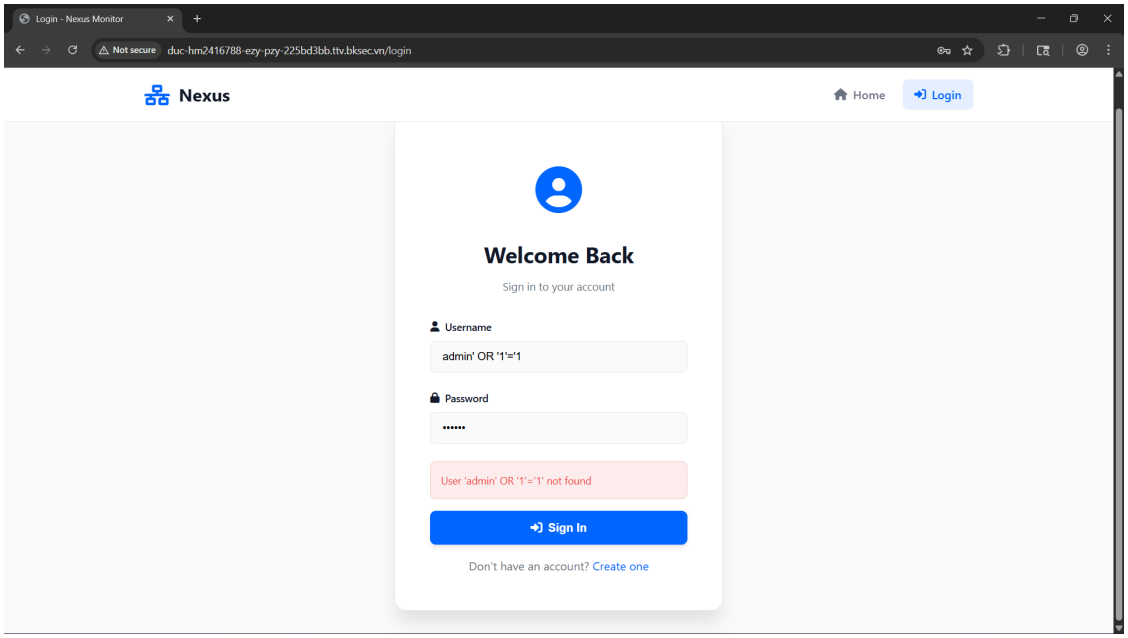
Để kiểm chứng điều này cũng như thử các hướng tấn công khác, em ném link này vào *Burp Suite* để bắt đầu vọc vạch.

2 Nhập payload và khảo sát Burp Suite

Em chèn vào ô *Username* một payload quen thuộc:

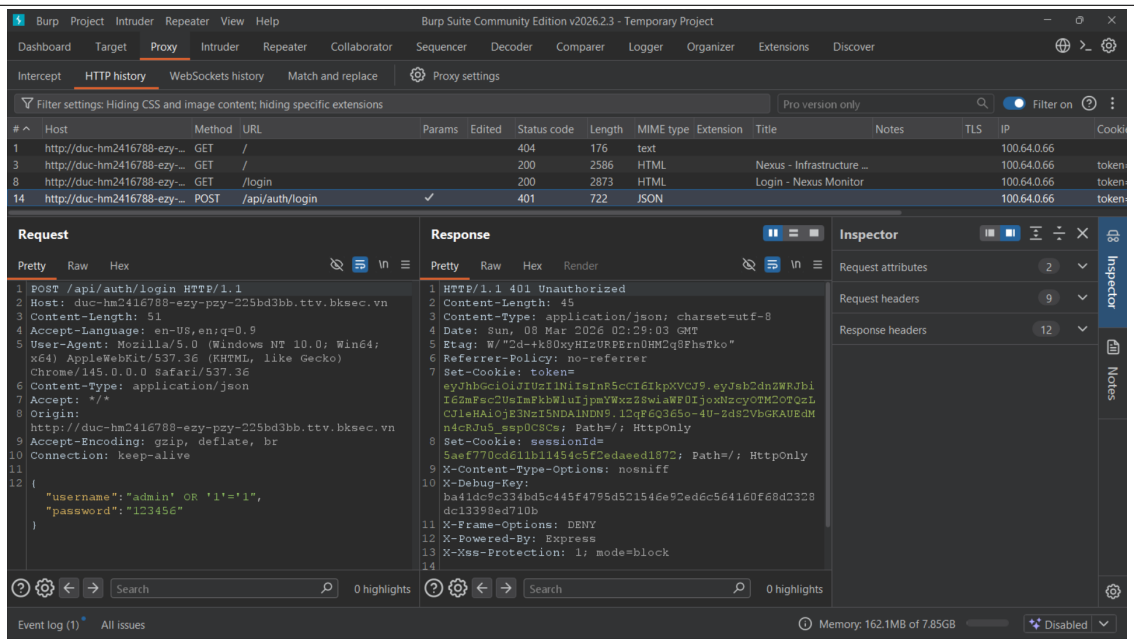
```
admin' OR '1'='1
```

còn *Password* thì điền bất kỳ. Kết quả là trang web trả về thông báo như sau:



Hình 3: Kết quả khi thử payload `admin' OR '1'='1` trên trang login.

Em mở phần *Response* của request này và thu được hai thông tin có giá trị:



Hình 4: Kết quả khi mở Burp Suite và đọc Response của request vừa rồi.

- **Set-Cookie:**

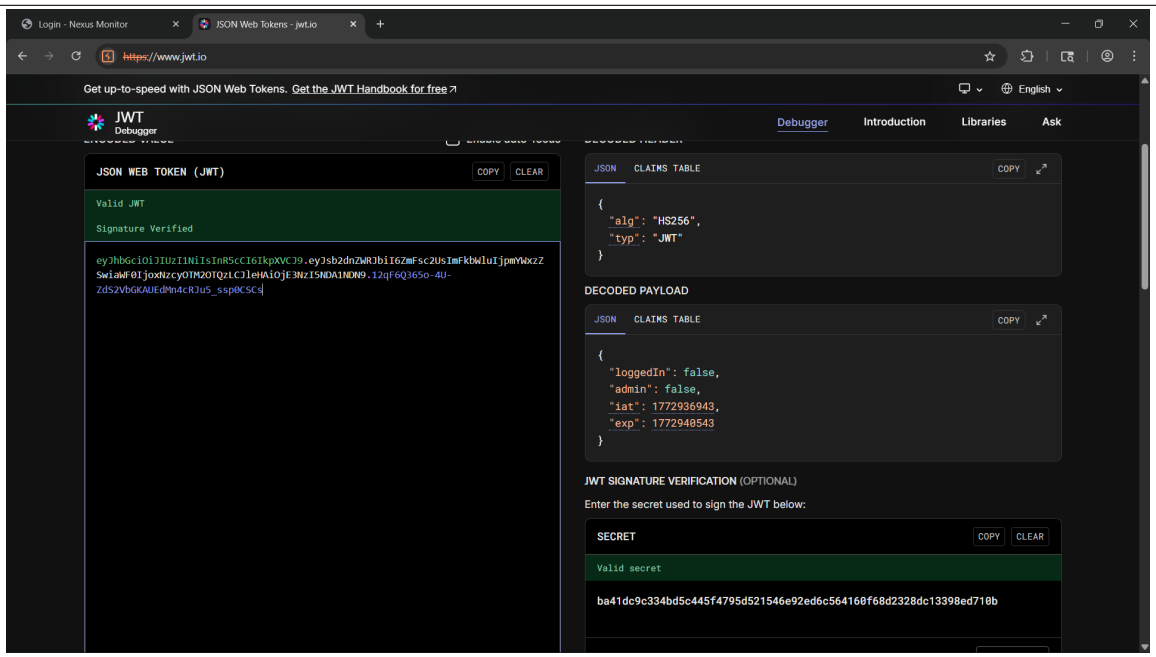
```
token=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJsb2dnZWRJbiI6ZmFsc2UsImFkbWluIjpmYWxzZSwiaWF0IjoxNzcyOTM2OTQzL
Path=/; HttpOnly
```

- **X-Debug-Key:**

```
ba41dc9c334bd5c445f4795d521546e92ed6c564160f68d2328dc13398ed710b
```

Dựa vào cấu trúc của `token`, có thể xác nhận rằng server sử dụng **JWT**. Ngoài ra, sự xuất hiện của header `X-Debug-Key` khiến em nghi ngờ rằng đây có thể chính là dữ liệu liên quan tới **key** dùng để ký *signature* của JWT.

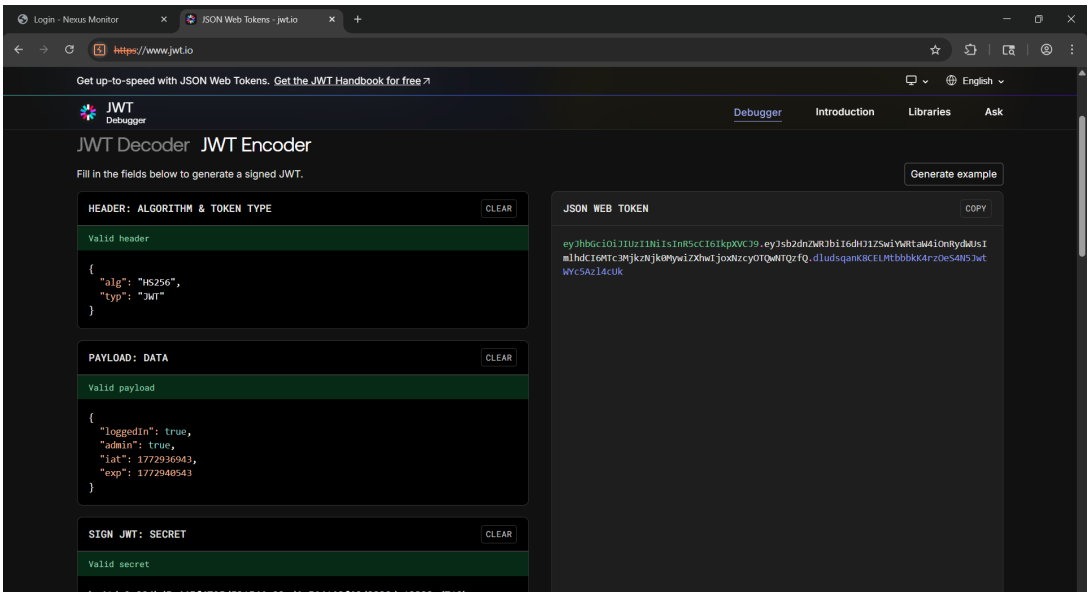
Vì vậy, em ném cả `token` và `X-Debug-Key` vào trang *jwt.io* và thu được dữ liệu sau khi decode:



Hình 5: Giải mã JWT trên jwt.io và thử kiểm tra signature bằng `X-Debug-Key`.

3 Thay đổi JWT để lấy quyền admin

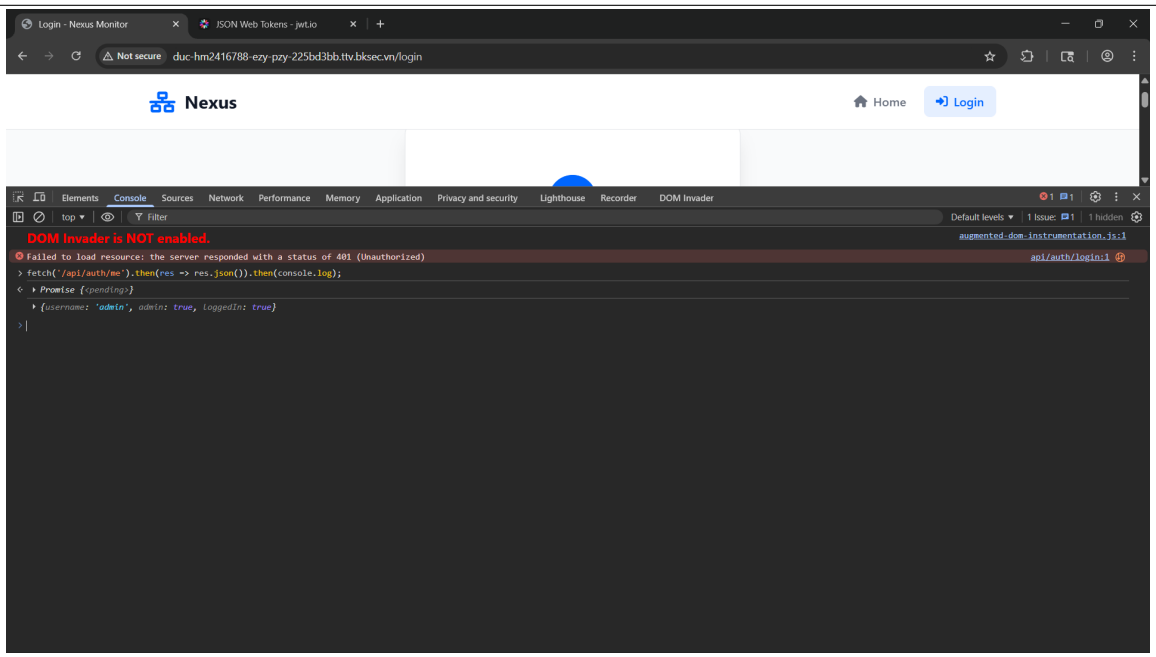
Em dùng luôn tính năng JWT Encoder để sử dụng key cũ lấy được từ X-Debug-Key, thay đổi các giá trị `loggedIn` và `admin` thành `true`, từ đó tạo ra 1 JWT mới.



Hình 6: Mã hoá lại header và payload mới bằng key đã tìm được.

Sau khi đã có token mới, em quay lại trang web, nhấn `F12` để mở Google DevTools → tab *Application* → *Cookies*, sau đó thay giá trị cookie `token` bằng JWT vừa tạo.

Để kiểm tra xem đã có quyền admin hay chưa, em chuyển sang tab *Console* và chạy lệnh kiểm tra quyền. Kết quả thu được như sau:

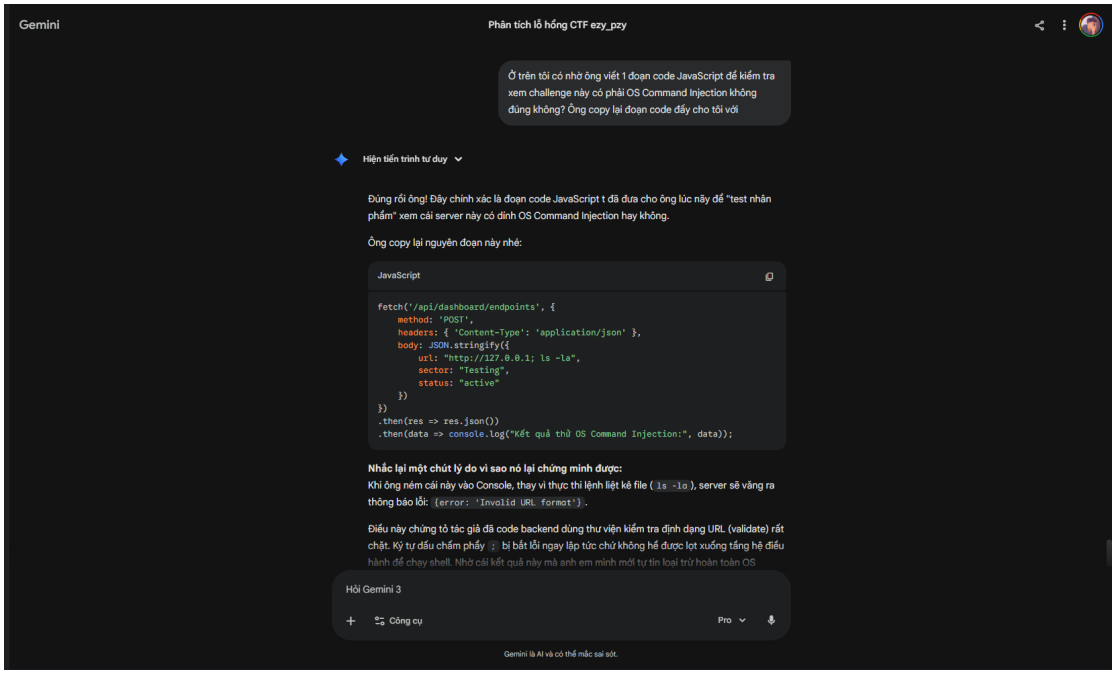


Hình 7: Kết quả kiểm tra sau khi thay JWT: `admin=true` và `loggedIn=true`.

4 Loại bỏ hướng OS Command Injection và tập trung vào SSRF

Sau khi đã nắm được quyền **admin**, em nhận định có hai hướng tấn công tiềm năng trong phần *dashboard* là **OS Command Injection** và **SSRF**.

Để kiểm tra liệu endpoint có dính **OS Command Injection** hay không, em có nhờ Gemini tạo một đoạn code JavaScript vào tab *Console* để gửi request tới API tạo endpoint (kèm payload thử chèn lệnh hệ điều hành):

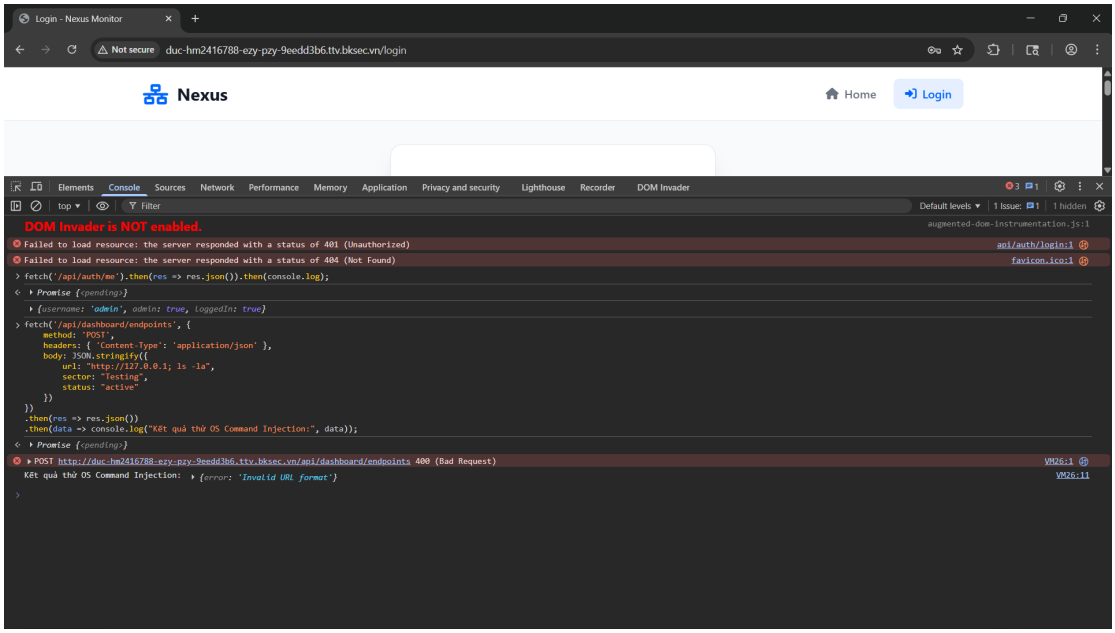


Hình 8: Ảnh em nhờ Gemini tạo 1 đoạn code để check (tại cái tin nhắn kia trôi hơi xa nên em hỏi như này cho nhanh).

```javascript
fetch('/api/dashboard/endpoints', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({
        url: "http://127.0.0.1; ls -la",
        sector: "Testing",
        status: "active"
    })
})
.then(res => res.json())
.then(data => console.log("Kết quả thử OS Command Injection:", data));
```

Hình 9: Đoạn JavaScript thử nghiệm OS Command Injection thông qua API `/api/dashboard/endpoints`.

Sau khi nhấn *Enter*, em thu được kết quả:



Hình 10: Kết quả thử nghiệm: server trả về lỗi `Invalid URL format`.

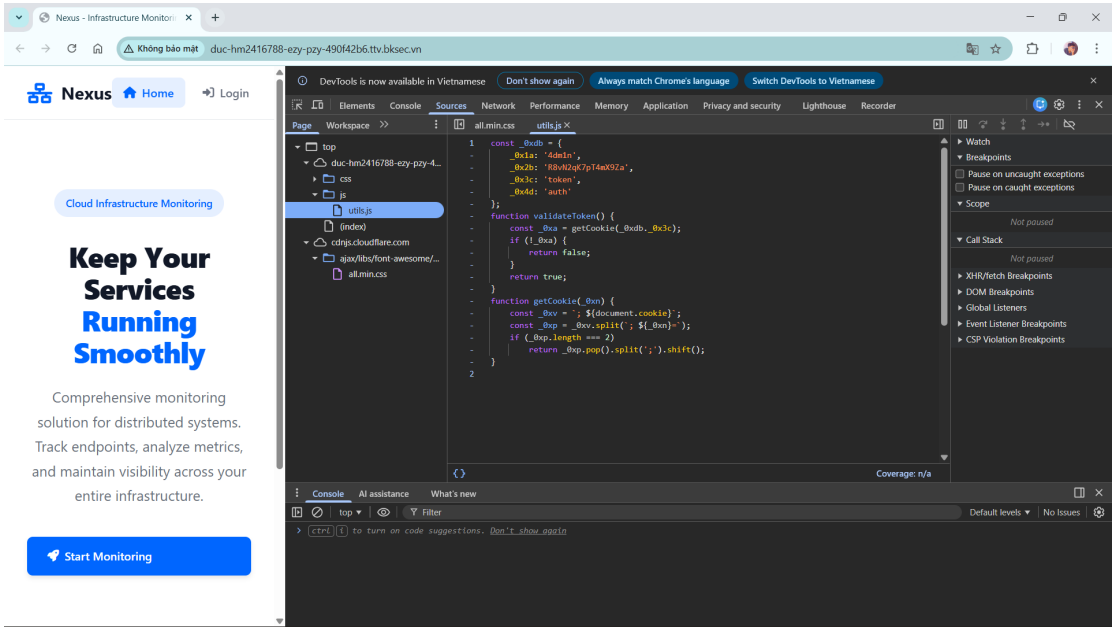
Rõ ràng server đang **validate chặt format URL** đầu vào. Trong khi đó, payload thử nghiệm của em chứa dấu `;` và dấu *space* - vốn là đặc trưng của việc nối lệnh trong Linux - nên bị chặn ngay từ lớp kiểm tra định dạng.

Vì vậy, em loại bỏ hướng tấn công **OS Command Injection** ở endpoint này và chuyển sang tập trung vào hướng khai thác hợp lý hơn: **SSRF**.

5 Khai thác SSRF để truy cập CouchDB và lấy flag

Để xác định database của server được lưu trữ theo dạng nào, em nhấn `F12` và vào tab *Sources*. Tại đây, trong folder `js` có file `utils.js` trông khá khả nghi. Khi mở file này, em phát hiện

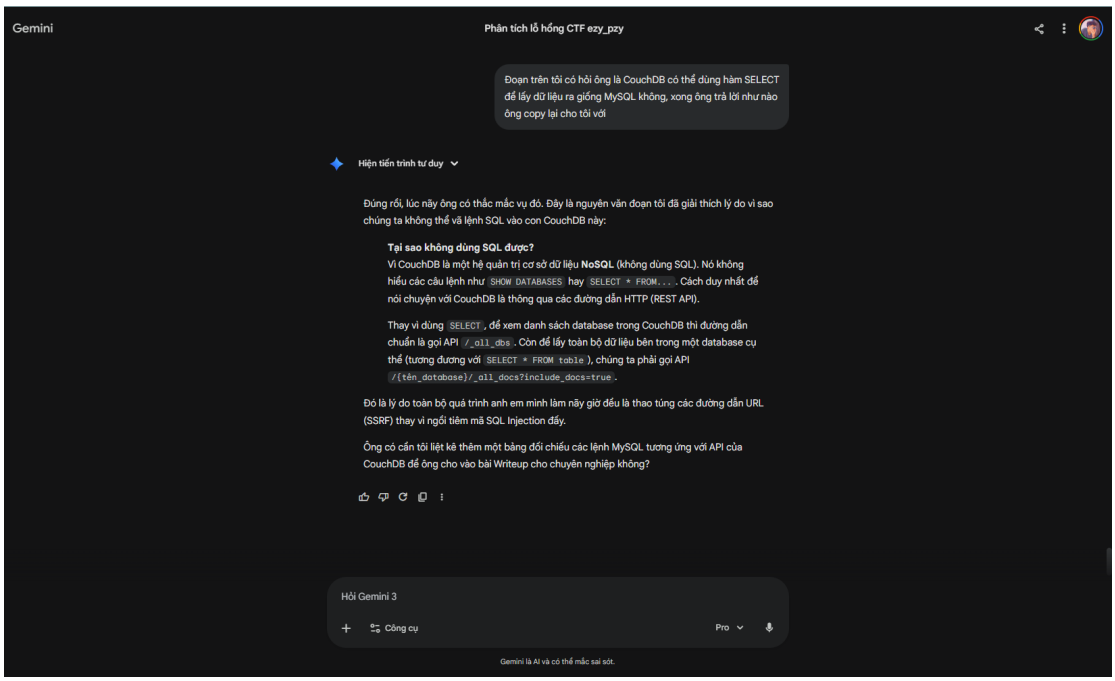
một giá trị đáng chú ý là _0x1a: 'c0uch_4dm1n' (Ban đầu nó là couch_4dm1n nhưng mà sau khi em solve được thì tác giả đã đổi lại thành 4dm1n).



Hình 11: Ảnh khi mở file utils.js ra xem.

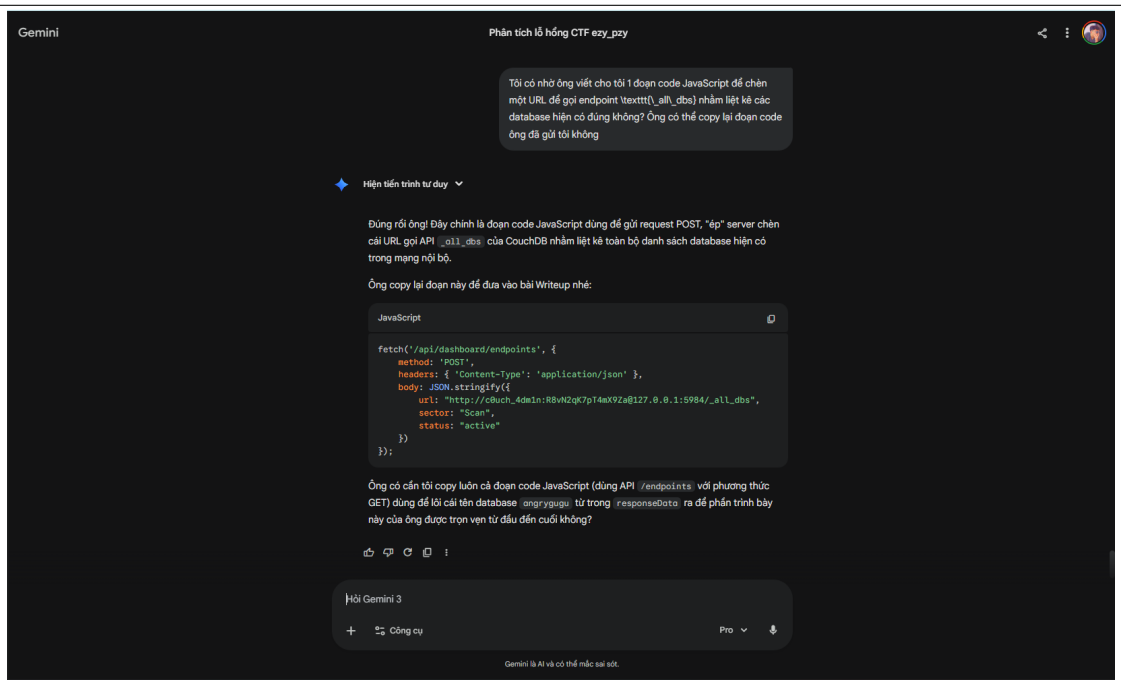
Chi tiết này cho thấy có vẻ trang web đang sử dụng **CouchDB** để lưu trữ dữ liệu.

Vì CouchDB là một database mà em chưa được học, nên em có hỏi Gemini cách để khai thác dữ liệu của nó, và đây là câu trả lời của Gemini:



Hình 12: Hỏi Gemini cách để khai thác dữ liệu của CouchDB.

Mặc định CouchDB thường chạy trên cổng `5984`, nên em nhờ Gemini viết 1 đoạn code JavaScript thử chèn một URL để gọi endpoint `_all_dbs` nhằm liệt kê các database hiện có:

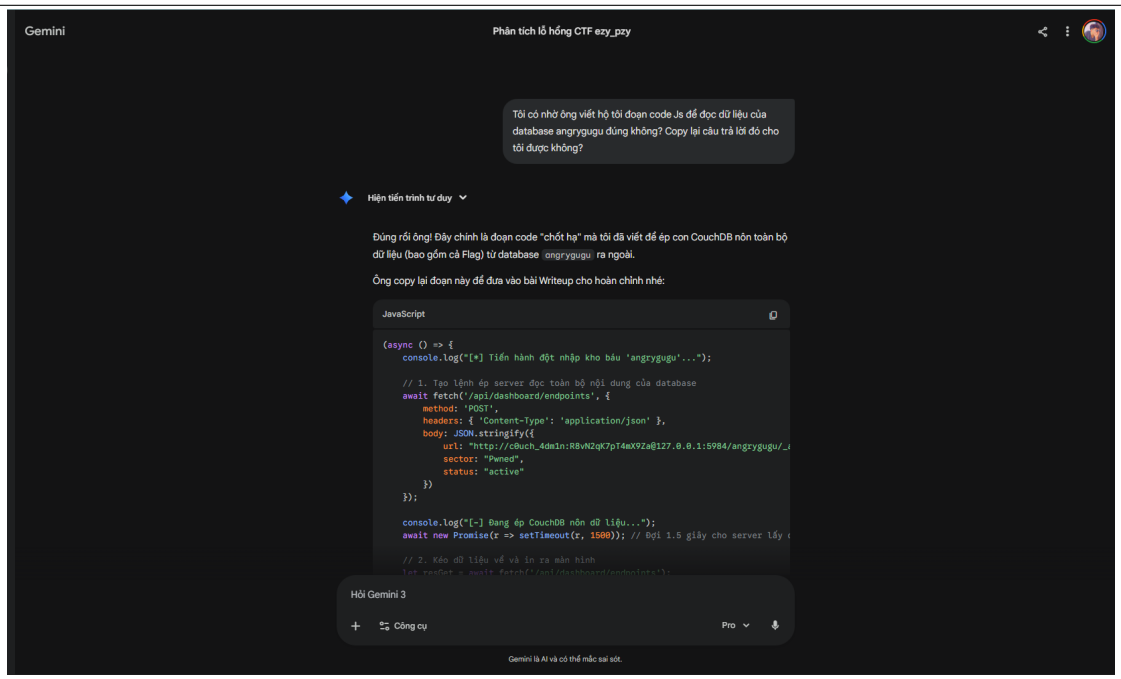


Hình 13: Nhờ Gemini viết 1 đoạn code Js để khai thác dữ liệu.

```
fetch('/api/dashboard/endpoints', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({
        url: "http://c0uch_4dm1n:R8vN2qK7pT4mX9Za@127.0.0.1:5984/_all_dbs",
        sector: "Scan",
        status: "active"
    })
});
```

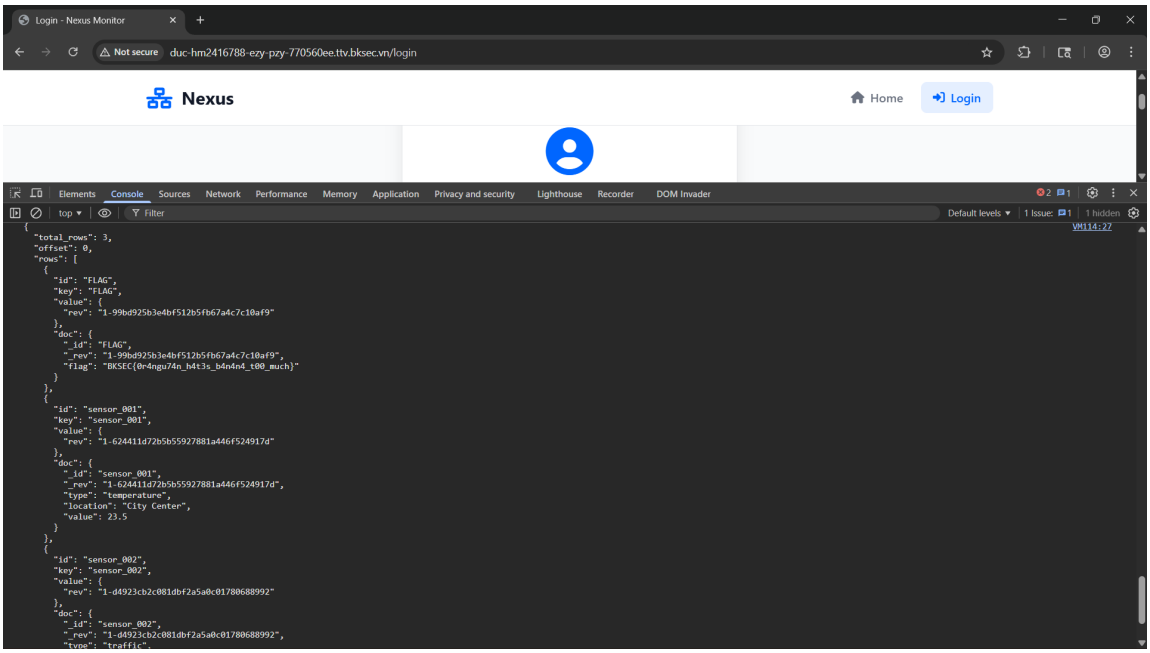
Hình 14: Khai thác SSRF để truy cập CouchDB qua `127.0.0.1:5984` và gọi `_all_dbs`.

Trang web đã trả về một database có tên là **angrygugu**. Chỉ chờ có thế, em tiếp tục nhờ Gemini viết code để đọc dữ liệu bên trong database **angrygugu**.



Hình 15: Đọc dữ liệu có trong database angrygugu.

Và đây là kết quả thu được:



Hình 16: Kết quả đọc dữ liệu từ database `angrygugu` và trích xuất được flag.

Em nhập thử flag được in ra, và cuối cùng xác định được **flag đúng** của bài này là:

`BKSEC{0r4ngu74n_h4t3s_b4n4n4_t00_much}`

Problem solved!